KET-RAG: A Cost-Efficient Multi-Granular Indexing Framework for Graph-RAG

Yiqian Huang
National University of Singapore
Singapore
yiqian@comp.nus.edu.sg

Shiqi Zhang*
National University of Singapore
Singapore
PyroWis AI
Singapore
shiqi@pyrowis.ai

Xiaokui Xiao
National University of Singapore
Singapore
xkxiao@nus.edu.sg

Abstract

Graph-RAG constructs a knowledge graph from text chunks to improve retrieval in Large Language Model (LLM)-based question answering. It is particularly useful in domains such as biomedicine, law, and political science, where retrieval often requires multi-hop reasoning over proprietary documents. Some existing Graph-RAG systems construct KNN graphs based on text chunk relevance, but this coarse-grained approach fails to capture entity relationships within texts, leading to sub-par retrieval and generation quality. To address this, recent solutions leverage LLMs to extract entities and relationships from text chunks, constructing triplet-based knowledge graphs. However, this approach incurs significant indexing costs, especially for large document collections.

To ensure a good result accuracy while reducing the indexing cost, we propose KET-RAG, a multi-granular indexing framework. KET-RAG first identifies a small set of key text chunks and leverages an LLM to construct a knowledge graph skeleton. It then builds a text-keyword bipartite graph from all text chunks, serving as a lightweight alternative to a full knowledge graph. During retrieval, KET-RAG searches both structures: it follows the local search strategy of existing Graph-RAG systems on the skeleton while mimicking this search on the bipartite graph to improve retrieval quality. We evaluate 13 solutions on three real-world datasets, demonstrating that KET-RAG outperforms all competitors in indexing cost, retrieval effectiveness, and generation quality. Notably, it achieves comparable or superior retrieval quality to Microsoft's Graph-RAG while reducing indexing costs by over an order of magnitude. Additionally, it improves the generation quality by up to 32.4% while lowering indexing costs by around 20%.

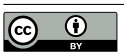
CCS Concepts

• **Information systems** → **Retrieval models and ranking**; • **Computing methodologies** → *Information extraction*; *Knowledge representation and reasoning*.

Keywords

Retrieval-Augmented Generation; GraphRAG; Indexing

*Corresponding author.



KDD '25, Toronto, ON, Canada

ACM ISBN 979-8-4007-1454-2/2025/08
https://doi.org/10.1145/3711896.3737012

ACM Reference Format:
Yiqian Huang, Shiqi Zhang, and Xiaokui Xiao. 2025. KET-RAG: A Cost-Efficient Multi-Granular Indexing Framework for Graph-RAG. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '25), August 3–7, 2025, Toronto, ON, Canada.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3711896.3737012

1 Introduction

Given a set of text chunks $\mathcal{T}$, Graph-based Retrieval-Augmented Generation (Graph-RAG) [8, 26] enhances generative model inference by structuring $\mathcal{T}$ into a Text-Attributed Graph (TAG) $\mathcal{G}$ and retrieving relevant information from it. Compared to Text-RAG [19], which retrieves independent text chunks from $\mathcal{T}$, Graph-RAG captures relationships within and across text snippets to enhance multi-hop reasoning [7, 17, 29]. Graph-RAG has gained widespread adoption across domains such as e-commerce [34, 36], biomedical research [7, 20], healthcare [4], political science [24], legal applications [5, 18], and many others [1, 30].

Some studies [21, 35] instantiate TAG $\mathcal{G}$ as a K-nearest-neighbor (KNN) graph, where nodes represent text chunks in $\mathcal{T}$, and edges encode semantic similarity or relevance. This approach maintains a low graph construction cost comparable to Text-RAG. However, it fails to capture entities and their relationships within text chunks, limiting retrieval effectiveness and degrading the quality of generated answers. To address this limitation, recent studies [7, 8, 14, 20] have turned to triplet-based knowledge graphs, leveraging Large Language Models (LLMs) to extract structured (entity, relation, entity) triplets from text. This approach, known as KG-RAG, enables the LLM to filter out noise in raw documents and construct a more structured and interpretable knowledge base, significantly improving retrieval and generation quality. As a result, it has gained significant traction by major companies, including Microsoft [8], Ant Group [11], Neo4j [27], and NebulaGraph [26]. However, KG-RAG comes with a high indexing cost, particularly for large datasets. Even with the cost-efficient GPT-4o-mini API, processing a 3.2MB sample of the HotpotQA dataset [37] costs $21. In real-world applications, textual data often spans gigabytes to terabytes, making indexing costs prohibitively expensive. For instance, processing a single 5GB legal case [2] incurs an estimated $33K, posing a significant challenge for large-scale adoption.

To improve retrieval and generation quality while lowering indexing costs, we propose KET-RAG, a cost-efficient multi-granular indexing framework for Graph-RAG. It comprises two key components: a knowledge graph skeleton and a text-keyword bipartite

graph. Instead of fully materializing the knowledge graph, KET-RAG first identifies a set of core text chunks from $\mathcal{T}$ based on their PageRank centralities [28] in an intermediate KNN graph. It then constructs a skeleton of the complete knowledge graph using KG-RAG described above. To prevent information loss from relying solely on this skeleton, KET-RAG also builds a text-keyword bipartite from $\mathcal{T}$, serving as a lightweight alternative to KG-RAG. By linking keywords to the text chunks in which they appear, keywords and their neighboring text chunks can be regarded as candidate entities and corresponding relations in the knowledge graph. During retrieval, KET-RAG adopts the local search strategy of existing solutions but, unlike previous methods, extracts ego networks from both entity and keyword channels to facilitate LLM-based generation.

In experiments, we evaluate 13 solutions on three datasets across three key aspects: indexing cost, retrieval quality, and generation quality. Notably, KET-RAG achieves retrieval quality comparable to or better than Microsoft's Graph-RAG [8], the state-of-the-art KG-RAG solution, while reducing indexing costs by over an order of magnitude. At the same time, it improves generation quality by up to 32.4% while lowering indexing costs by approximately 20%. Furthermore, the core components of KET-RAG, Skeleton-RAG and Keyword-RAG, also function as effective stand-alone RAG solutions, balancing efficiency and quality. In particular, Skeleton-RAG reduces indexing costs by 20% while maintaining retrieval quality, showing only minor performance drops in low-cost settings and achieving parity or even slight improvements in high-accuracy configurations. Meanwhile, Keyword-RAG consistently outperforms the vanilla Text-RAG in both retrieval and generation quality, achieving up to 92.4%, 133.3%, 118.5%, and 12.9% relative improvements in Coverage, EM, F1 scores, and BERTScore, respectively, while exhibiting no compromise in retrieval latency.

To summarize, we make the following contributions in this work:

- We propose KET-RAG, a cost-efficient multi-granular indexing framework for Graph-RAG, integrating two complementary components to balance indexing cost and result quality.
- We introduce Skeleton-RAG, which constructs a knowledge graph skeleton by selecting core text chunks and leveraging LLMs to extract structured knowledge.
- We develop Keyword-RAG, a lightweight text-keyword bipartite graph that mimics the retrieval paradigm of KG-RAG while significantly reducing indexing costs.
- We conduct extensive experiments demonstrating the improvements of our proposed solutions.

2 Preliminaries

We first define the key terminologies and notations in Section 2.1, and present the objective of this work in Section 2.2. Finally, we review the state-of-the-art Microsoft's Graph-RAG in Section 2.3.

2.1 Terminologies and Notations

Let $\mathcal{T}$ denote a set of text chunks, which is preprocessed from a set of documents. For simplicity, we assume that each text chunk $t_i \in \mathcal{T}$ is partitioned into chunks of equal length, denoted by ℓ tokens. The text embedding of each chunk t_i is denoted as $\boldsymbol{\phi}(t_i)$. We define a *text-attributed graph* (TAG) index as $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ and $\mathcal{E}$ are the sets of nodes and edges, respectively. In $\mathcal{V}$, each node is represented as $v_i = (t_i, \boldsymbol{\phi}(t_i))$, where t_i is the textual attribute and $\boldsymbol{\phi}(t_i)$ is its text embedding. Analogously, each edge $e_{i,j} \in \mathcal{E}$ between nodes v_i and v_j is denoted as $e_{i,j} = (v_i, v_j, t_{i,j}, \boldsymbol{\phi}(t_{i,j}))$ if it is text-attributed; otherwise, it is simply $e_{i,j} = (v_i, v_j)$. In addition, we use calligraphic uppercase letters (e.g., $\mathcal{S}$) to denote sets of nodes or edges. For text information, $\boldsymbol{\phi}(\cdot)$ represents the text embedding function, the function $\bigoplus(\mathcal{S})$ represents the concatenation of all text in $\mathcal{S}$, and $\left|\bigoplus(\mathcal{S})\right|$ denotes the token count of the concatenated string. The frequently-used notations are summarized in Table 1.

Table 1: Frequently used notations.

Notation	Description
$\mathcal{T}, t_i, \ell$	Text chunk set $\mathcal{T}$ with each chunk t_i in length ℓ.
$\boldsymbol{\phi}(t_i)$	The text embedding of text t_i.
$\mathcal{G} = (\mathcal{V}, \mathcal{E})$	Node set $\mathcal{V}$ and edge set $\mathcal{E}$ in TAG index $\mathcal{G}$.
$\bigoplus(\mathcal{S}), \left\|\bigoplus(\mathcal{S})\right\|$	the concatenated texts from $\mathcal{S}$ and its token count.
K, β, τ	the integer K of KNN graph, the budget ratio β, and the number of splits τ.
λ, θ	the context limit λ and the retrieval ratio θ.

2.2 Problem Formulation

The Retrieval-Augmented Generation (RAG) framework consists of three main stages: indexing, retrieving, and generation.

Given a set of text chunks $\mathcal{T}$, the indexing stage of Text-RAG [19] generates a text embedding $\boldsymbol{\phi}(t_i)$ for each $t_i \in \mathcal{T}$. During the retrieval stage, Text-RAG computes the query embedding $\boldsymbol{\phi}(q)$ and retrieves text chunks from $\mathcal{T}$ that are most similar to the query in the embedding space. Finally, the retrieved text chunks are incorporated into a predefined prompt for a large language model (LLM) to generate the final response. In contrast, Graph-RAG constructs a TAG index $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ from $\mathcal{T}$ during indexing. Existing methods primarily build two types of $\mathcal{G}$: (i) a K-Nearest-Neighbors (KNN) graph (KNNG-RAG), where each text chunk is a node, and edges represent similarity-based connections [21, 35]; or (ii) a knowledge graph (KG-RAG), where LLMs extract (entity, relation, entity) triplets from text [7, 8, 14, 20]. During retrieval, Graph-RAG computes $\boldsymbol{\phi}(q)$ and identifies *seed nodes* in $\mathcal{G}$ that have the most similar text embeddings to the query. A subgraph is then extracted via local search, serialized into natural language, and incorporated into a predefined prompt for LLM-based response generation.

Our objective. This work aims to develop a cost-efficient and effective approach for Graph-RAG to construct a TAG index $\mathcal{G}$ from $\mathcal{T}$, achieving lower indexing costs yet higher result accuracy in the widely adopted local search scenario [7, 8, 14, 20].

2.3 Microsoft's Graph-RAG

Microsoft proposed the Graph-RAG system [8], which constructs a knowledge graph index with multi-level communities and employs tailored strategies for both local and global search. In this section, we focus on its indexing and retrieval operations for local search, which are relevant to our work.

Algorithm 1 outlines the pseudo-code for constructing the graph index $\mathcal{G} = (\mathcal{V}_e \cup \mathcal{V}_t, \mathcal{E})$, where $\mathcal{V}_e$ and $\mathcal{V}_t$ represent entities and text chunks, respectively. Given a text chunk set $\mathcal{T}$, KG-Index first

Algorithm 1: KG-Index ($\mathcal{T}$)

Input: The text chunk set $\mathcal{T}$.
Output: A TAG index $\mathcal{G}$.
1 $\mathcal{V}_t \leftarrow \{(t_i, \boldsymbol{\phi}(t_i)) \mid t_i \in \mathcal{T}\}$; $\mathcal{V}_e \leftarrow \emptyset$; $\mathcal{E} \leftarrow \emptyset$;
2 **for** *each* $v_i = (t_i, \boldsymbol{\phi}(t_i)) \in \mathcal{V}_t$ **do**
3 $\quad \mathcal{V}_i$, $\mathcal{E}_i \leftarrow$ entities and edges extracted by LLM from t_i;
4 $\quad \mathcal{V}_e \leftarrow \mathcal{V}_e \cup \mathcal{V}_i$; $\mathcal{E} \leftarrow \mathcal{E} \cup \mathcal{E}_i$;
5 $\quad \mathcal{E} \leftarrow \mathcal{E} \cup \{(v_i, x) \mid x \in (\mathcal{V}_i \cup \mathcal{E}_i)\}$;
6 **return** $(\mathcal{V}_e \cup \mathcal{V}_t, \mathcal{E})$;

Algorithm 2: KG-Retrieval ($\mathcal{G}, q, \lambda$)

Input: A TAG index $\mathcal{G} = (\mathcal{V}_e \cup \mathcal{V}_t, \mathcal{E})$, a query q, context length limit λ
Output: The context C
1 $\mathcal{S}_e \leftarrow \underset{v_i \in \mathcal{V}_e}{\arg\min\text{-k}}\ \mathrm{euc}(v_i, q)$, where $k = 10$;
2 $\mathcal{S}_r \leftarrow \underset{e_{i,j} \in \mathcal{E}}{\arg\max\text{-k}}\ \mathrm{adj}(\mathcal{S}_e, e_{i,j})$, s.t. $|\bigoplus(\mathcal{S}_r)|+|\bigoplus(\mathcal{S}_e)|=\lambda/2$;
3 $\mathcal{S}_t \leftarrow \underset{v_i \in \mathcal{V}_t}{\arg\max\text{-k}}\ \mathrm{adj}(\mathcal{S}_e \cup \mathcal{S}_r, v_i)$, s.t. $|\bigoplus(\mathcal{S}_t)|=\lambda/2$;
4 **return** $\bigoplus(\mathcal{S}_e \cup \mathcal{S}_r \cup \mathcal{S}_t)$;

generates a text embedding for each $t_i \in \mathcal{T}$ and treats the corresponding text snippets as text-attributed nodes, forming the node set $\mathcal{V}_t$ (Line 1). For each node $v_i \in \mathcal{V}_t$, KG-Index leverages a predefined LLM to process t_i in two steps: *entity identification* and *relationship extraction*, obtaining the sets of entities and relations ($\mathcal{V}_i$ and $\mathcal{E}_i$) (Line 3). These extracted entities and relationships are then added to $\mathcal{V}_e$ and $\mathcal{E}$, respectively (Line 4). For each extracted entity or relationship $x \in (\mathcal{V}_i \cup \mathcal{E}_i)$, the LLM generates a textual description t_x along with its embedding $\boldsymbol{\phi}(t_x)$. Additionally, KG-Index links each text chunk node v_i to its corresponding extracted entities and relationships (Line 5).

The local search procedure retrieves context from entities, relationships, and text chunks. Algorithm 2 outlines the retrieval process, following the default context limit ratio across channels. Given a graph index $\mathcal{G}$ and a context limit λ, KG-Retrieval retrieves contexts in the order of seed entities, relationships, and text chunks. Specifically, it first retrieves 10 entity nodes ($\mathcal{S}_e$) with embeddings most similar to the query embedding based on Euclidean distance (Line 1). It then retrieves relationships ($\mathcal{S}_r$) until the combined token count from $\mathcal{S}_e \cup \mathcal{S}_r$ reaches $\lambda/2$ (Line 2). Relationships are prioritized based on adjacency to $\mathcal{S}_e$, with those connecting two seed entities ranking higher. For text chunk retrieval, KG-Retrieval retrieves text chunks most adjacent to $\mathcal{S}_e$ and $\mathcal{S}_r$ until the total context length reaches λ (Line 3). At last, the retrieved texts from $\mathcal{S}_e \cup \mathcal{S}_r \cup \mathcal{S}_t$ are concatenated to form the final context, which is returned as input for generation (Line 4).

3 Related Works

Beyond Microsoft's Graph-RAG, we review other indexing and retrieval approaches within existing Graph-RAG frameworks. For a comprehensive review, we refer readers to surveys [9, 29, 39].

Given $\mathcal{T}$, the core of constructing a KNN graph is measuring text chunk similarity. In particular, Li et al. [21] consider both structural and lexical similarities. Structural similarity is based on the physical adjacency of text chunks, linking neighboring passages in $\mathcal{G}$. Lexical similarity connects chunk nodes that share common keywords, which are extracted using LLM-based prompting. Wang et al. [35] leverage multiple lexical similarity measures. Two chunk nodes are connected if they share keywords extracted using TF-IDF [31], contain common Wikipedia entities identified via TAGME [23], or exhibit semantic similarity based on text embeddings. Recent works [7, 8, 14, 20] use LLMs to extract (entity, relation, entity) triplets from $\mathcal{T}$ to build knowledge graph indices, improving retrieval quality. Akin to Algorithm 1, Delile et al. [7] employ PubmedBERT [12] to extract triplets from biomedical texts and link entities to the text chunks in which they appear. Gutierrez et al. [14] further enrich graph connectivity by linking semantically similar entities within the knowledge graph. Several studies construct TAG indices using explicit relationships, such as co-authorship or citation links in academic papers [25], trade relationships between companies [3], and other structured connections [17]. These publicly curated indices are beyond the scope of this work. In summary, KNNG-RAG is a more cost-effective solution but lacks fine-grained entity relationships. In contrast, KG-RAG achieves higher effectiveness but incurs significant indexing costs, particularly for large $\mathcal{T}$.

To retrieve the most relevant subgraph given a query, various local search strategies have been proposed. Below, we focus on methods that utilize heuristic or traditional graph algorithms. Similar to Algorithm 2, Jin et al. [17] extract ego networks from seed nodes to enhance retrieval precision. In addition, Li et al. [20] propose a two-step approach that first extracts a k-hop subgraph from seeds, followed by reranking and pruning the subgraph using LLMs. Other approaches include shortest path retrieval, where Delile et al. [7] and Mavromatis and Karypis [22] retrieve the shortest path between seed nodes. Gutierrez et al. [14] use personalized PageRank to extract relevant subgraphs. G-Retriever [16] focuses on query-aware subgraph generation by balancing semantic similarity to the query with subgraph generation cost. Hybrid retrieval methods, such as Hybrid-RAG [32] and EWEK-QA [6], enhance retrieval by querying both text and knowledge graphs.

4 Proposed Framework: KET-RAG

To fully leverage the strengths of existing graph indices while addressing their limitations, we introduce KET-RAG, an indexing framework that integrates multiple levels of granularity: Keywords, Entities, and Text chunks, into the TAG index $\mathcal{G}$. The overall workflow of KET-RAG is illustrated in Figure 1.

4.1 Overview

At a high level, the TAG index $\mathcal{G} = \mathcal{G}_s \cup \mathcal{G}_k$ consists of: (i) a knowledge graph *skeleton* $\mathcal{G}_s$, derived from a selected set of important text chunks called *core chunks*, and (ii) a text-keyword bipartite graph $\mathcal{G}_k$, constructed from all chunks. As shown in Figure 1, the construction process involves three main steps.

(1) KET-RAG first organizes the input text chunks in $\mathcal{T}$ into a KNN graph, where chunks are linked if they exhibit sufficient lexical

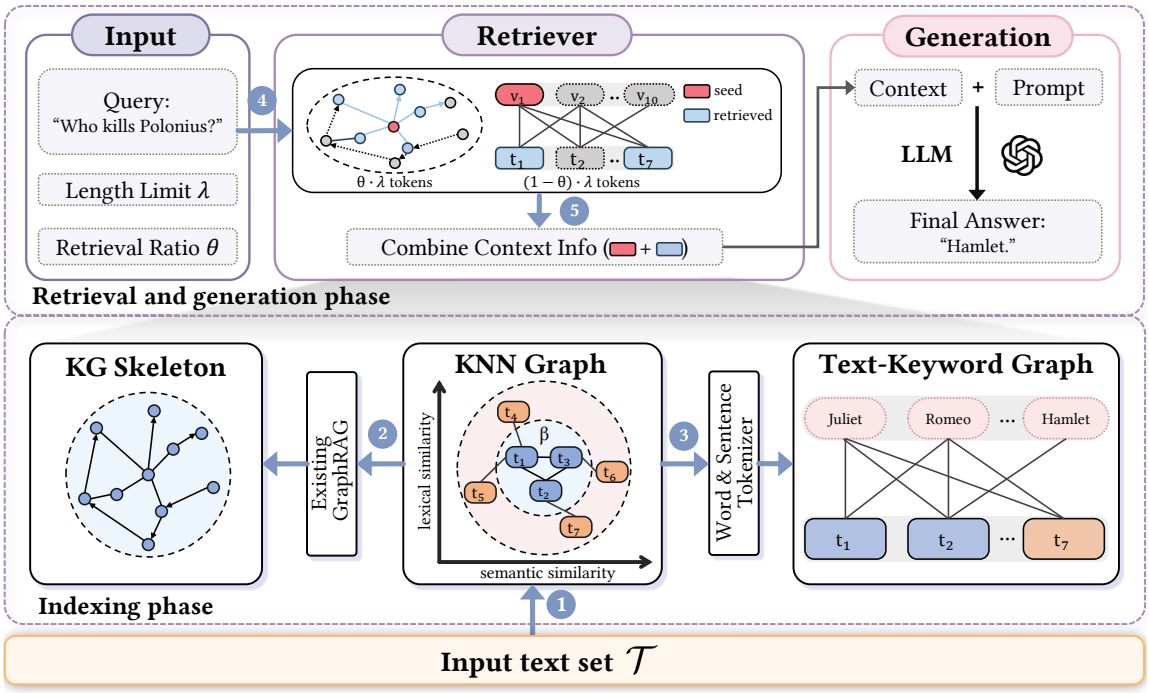


Figure 1: The illustration of KET-RAG: the indexing stage in ①-③ and the retrieval stage in ④-⑤.

or semantic similarity. This serves as an intermediate structure for building the final graph $\mathcal{G}$.

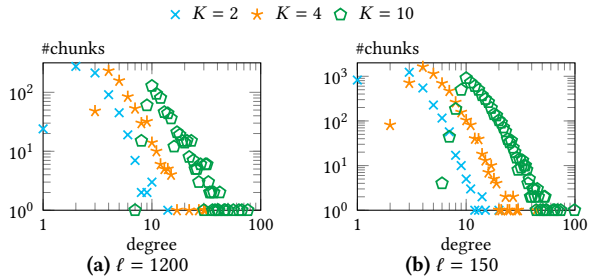
(2) Next, KET-RAG selects a β fraction of *core chunks* according to their structural importance in the KNN graph. These core chunks are then processed using KG-Index (Algorithm 1) to produce a knowledge graph skeleton $\mathcal{G}_s$.

(3) Finally, KET-RAG constructs the bipartite graph $\mathcal{G}_k = (\mathcal{V}_k \cup \mathcal{V}_t, \mathcal{E}_k)$ from $\mathcal{T}$. In $\mathcal{G}_k$, the node set $\mathcal{V}_k$ represents keywords, and $\mathcal{V}_t$ represents text chunks. An edge $e_{i,j} \in \mathcal{E}_k$ indicates that keyword node v_i appears in text chunk node v_j. Each keyword node v_i is assigned a description t_i (consisting of all sentences containing that keyword), and its embedding $\boldsymbol{\phi}(t_i)$ is computed as the average of these sentences' embeddings.

During retrieval, KET-RAG balances information from $\mathcal{G}_s$ and $\mathcal{G}_k$ using a constant θ. It first identifies a set of *seed nodes*, either entities or keywords, that are most similar to the query q in the text embedding space. For entity seeds, KET-RAG applies Algorithm 2 to retrieve context using θ proportion of the total context limit λ. For keyword seeds, it follows a similar procedure to collect relevant neighboring text chunks using the remaining $(1-\theta)$ of the context budget. Finally, the retrieved context is combined with a predefined prompt and passed to the LLM for response generation.

4.2 Rationale and Comparison

KET-RAG is motivated by two key observations. First, a small subset of core text chunks often exhibits broad relevance to others. Figure 2 presents the degree distribution of the KNN graph constructed from the MuSiQue dataset with input chunk sizes $\ell = 1200$ and $\ell = 150$. This heavily skewed distribution highlights the importance of core chunks in linking different parts of the graph. Consequently, these



(a) $\ell = 1200$ **(b)** $\ell = 150$

Figure 2: Log-log Plot of the degree distribution of the KNN graph on MuSiQue.

core chunks should be prioritized to extract high-quality triplets using the LLM. Second, in the lightweight alternative graph $\mathcal{G}_k$, keywords and their neighboring text chunks can serve as stand-ins for entities and their ego networks. Specifically, when seed keywords align with seed entities, their neighboring text chunks are expected to contain information about those entities' ego networks. Hence, these neighboring chunks are treated as candidates, and retrieval follows the standard Text-RAG strategy.

To summarize, compared to previous KG-RAG solutions [7, 8, 14, 20], KET-RAG focuses on a smaller set of core chunks to construct a knowledge graph skeleton while leveraging a text-keyword bipartite graph as a lightweight alternative. This design lowers the cost of LLM inference and improves result quality via two distinct retrieval channels (entity and keyword). Additionally, in the keyword channel, KET-RAG confines the retrieval to snippets containing seed keywords, unlike Text-RAG, which searches across the entire $\mathcal{T}$.

Algorithm 3: KET-Index $(\mathcal{T}, K, \beta, \tau)$

Input: The text chunk set $\mathcal{T}$, an integer K, a budget rate β, the number of splits τ.
Output: A TAG index $\mathcal{G}$.

1 $\mathcal{W} \leftarrow$ all keywords tokenized from $\mathcal{T}$;
2 $\mathcal{V} \leftarrow \{v_i = (t_i, \boldsymbol{\phi}(t_i)) \mid t_i \in \mathcal{T}\}$; $\mathcal{E} \leftarrow \emptyset$;
3 **for** *each* $v_i \in \mathcal{V}$ **do**
4 $\quad \mathcal{S}_1 \leftarrow \underset{v_j \in \mathcal{V} \setminus \{v_i\}}{\arg\max\text{-k}}$ co-occ(v_i, v_j), where $k = K/2$;
5 $\quad \mathcal{S}_2 \leftarrow \underset{v_j \in \mathcal{V} \setminus (\mathcal{S}_1 \cup \{v_i\})}{\arg\max\text{-k}}$ $\cos(v_i, v_j)$, where $k = K/2$;
6 $\quad \mathcal{E} \leftarrow \mathcal{E} \cup \{(v_i, v_j) \mid v_j \in (\mathcal{S}_1 \cup \mathcal{S}_2)\}$;
7 $\mathcal{G} \leftarrow (\mathcal{V}, \mathcal{E})$;
8 $\mathcal{V}_c \leftarrow \underset{v_i \in \mathcal{V}}{\arg\max\text{-k}}\ \pi_i$ in Eq. (1), where $k = \lceil \beta \cdot |\mathcal{V}| \rceil$;
9 $\mathcal{G}_s \leftarrow$ KG-Index $(\mathcal{V}_c)$ in Algorithm 1;
10 $\mathcal{T}_\tau \leftarrow$ split each $t_i \in \mathcal{T}$ into 2^τ equal-sized sub-chunks;
11 $\mathcal{V}_t \leftarrow \{v_i = (t_i, \boldsymbol{\phi}(t_i)) \mid t_i \in \mathcal{T}_\tau\}$; $\mathcal{V}_k \leftarrow \emptyset$; $\mathcal{E}_k \leftarrow \emptyset$;
12 **for** *each keyword* $x \in \mathcal{W}$ **do**
13 $\quad \mathcal{V}_k \leftarrow \mathcal{V}_k \cup \{v_j = (t_j, \boldsymbol{\phi}(t_j))\}$, where t_j is all sentences in $\mathcal{T}_\tau$ containing x and $\boldsymbol{\phi}(t_j)$ is the average embedding;
14 $\quad \mathcal{E}_k \leftarrow \mathcal{E}_k \cup \{(v_i, v_j) \mid v_i \in \mathcal{V}_t,\ v_j \in \mathcal{V}_k,\ t_i \text{ contains } x\}$;
15 $\mathcal{G}_k \leftarrow (\mathcal{V}_k \cup \mathcal{V}_t, \mathcal{E}_k)$;
16 **return** $\mathcal{G}_s \cup \mathcal{G}_k$;

This subgraph-based approach better captures in-text relationships w.r.t. seed keywords, enhancing overall effectiveness.

5 Detailed Implementations

This section provides a detailed explanation of KET-RAG, with the indexing stage KET-Index discussed in Section 5.1 and the retrieval process KET-Retrieval described in Section 5.2.

5.1 KET-Index

As outlined in Algorithm 3, KET-Index takes as input a set $\mathcal{T}$ of text chunks, an integer K for the KNN graph, a budget rate β, and an integer τ for text chunk splitting. It first tokenizes all text chunks in $\mathcal{T}$ into vocabulary $\mathcal{W}$. By default, it tokenizes chunks into words while excluding stop words (e.g., 'the', 'a', and 'is') to define keyword nodes, though traditional keyword extraction methods [31] can also be applied. KET-Index then executes three core subroutines.

KNN Graph Initialization. In Lines 2–7, KET-Index represents each text chunk $t_i \in \mathcal{T}$ with its embedding $\boldsymbol{\phi}(t_i)$ as a node $v_i \in \mathcal{V}$. It then links each node v_i to the top-$K/2$ nodes based on lexical similarity and the top-$K/2$ nodes based on semantic similarity, forming the KNN graph $\mathcal{G}$ (Lines 3–6). Specifically, the lexical similarity between nodes v_i, v_j is defined as the number of co-occurring keywords in $\mathcal{W}$, while the semantic similarity is measured using the cosine similarity between their embeddings $\boldsymbol{\phi}(t_i)$ and $\boldsymbol{\phi}(t_j)$.

core chunk identification. Motivated by the observations in Figure 2, given an intermediate KNN graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ and a budget rate $\beta \in [0, 1]$, Line 8 of Algorithm 3 selects a set $\mathcal{V}_c$ of $\lceil \beta \cdot |\mathcal{V}| \rceil$ core chunk nodes based on their structural importance. For each node $v_i \in \mathcal{V}$, the PageRank value [28] π_i serves as a measure of structural importance. Let $\mathbf{P}$ be the probability transition matrix of $\mathcal{G}$, where $\mathbf{P}_{i,j} = \frac{1}{d(v_i)}$ and $d(v_i)$ denotes the degree of v_i. Given a teleport probability α, the PageRank vector $\boldsymbol{\pi}$ is computed as:

$$\boldsymbol{\pi} = \alpha \cdot \mathbf{1}/\mathbf{n} + (1 - \alpha) \cdot \boldsymbol{\pi} \cdot \mathbf{P}, \tag{1}$$

where $\mathbf{1}/\mathbf{n}$ is the initial vector with each of $n = |\mathcal{V}|$ dimensions set to $1/n$, and $\pi_i = \boldsymbol{\pi}[i]$ represents the PageRank score of $v_i \in \mathcal{V}$. PageRank effectively captures both direct and higher-order structural importance, making it suitable for identifying core chunks.

Graph Index Construction. In Line 9, KET-Index processes the selected core text chunks $\mathcal{V}_c$ using KG-Index (Algorithm 1) to construct the knowledge graph skeleton $\mathcal{G}_s$. Since the text-attributed node set $\mathcal{V}_c$ has already been built, Line 1 in Algorithm 1 is skipped. Next, in Lines 10–15, KET-Index constructs a text-keyword bipartite graph $\mathcal{G}_k = (\mathcal{V}_k \cup \mathcal{V}_t, \mathcal{E}_k)$ based on $\mathcal{T}$. Specifically, each text chunk in $\mathcal{T}$ is recursively divided into equal-sized sub-chunks over τ iterations, forming a set $\mathcal{T}_\tau$ with $2^\tau \cdot |\mathcal{T}|$ sub-chunks. Each sub-chunk is then initialized to a node in $\mathcal{V}_t$. For each keyword $x \in \mathcal{W}$, KET-Index creates a keyword node $v_j = (t_j, \boldsymbol{\phi}(t_j))$, where t_j concatenates all sentences in $\mathcal{T}$ containing x, and $\boldsymbol{\phi}(t_j)$ is their average embedding. This process aggregates information from different contexts, reducing noise and generating a more generalized representation of the keyword. Finally, KET-Index links each chunk node $v_i \in \mathcal{V}_t$ to its corresponding keyword node v_j. After constructing $\mathcal{G}_k$, KET-Index returns $\mathcal{G}_s \cup \mathcal{G}_k$ as the final index $\mathcal{G}$, where text chunk nodes in $\mathcal{G}_s$ are replaced by their corresponding partitioned sub-chunk nodes in $\mathcal{V}_t$ of $\mathcal{G}_k$, with edges in $\mathcal{G}_s$ rewired accordingly.

5.2 KET-Retrieval

As outlined in Algorithm 4, KET-Retrieval takes as input a TAG index $\mathcal{G} = \mathcal{G}_s \cup \mathcal{G}_k$, a query q, a context length limit λ, and a retrieval ratio θ. It outputs a context C by selecting the most relevant content from the skeleton graph $\mathcal{G}_s$ and the keyword-text bipartite graph $\mathcal{G}_k$. In Line 1, KET-Retrieval invokes KG-Retrieval (Algorithm 2) to retrieve context C_s from $\mathcal{G}_s$, using $\theta \cdot \lambda$ tokens. Next, in Lines 2–4, it retrieves content from $\mathcal{G}_k$ using the remaining $(1 - \theta) \cdot \lambda$ tokens. Specifically, KET-Retrieval iteratively selects seed keyword nodes $\mathcal{S}_k$ from $\mathcal{V}_k$ based on cosine similarity to the query embedding, expanding the selection until their neighboring text sub-chunks contain $2 \cdot (1 - \theta) \cdot \lambda$ tokens, i.e., $\left|\bigoplus (\mathcal{N}(\mathcal{S}_k))\right| = 2 \cdot (1 - \theta) \cdot \lambda$. Focusing on the candidate set $\mathcal{N}(\mathcal{S}_k)$, KET-Retrieval retrieves the final text set $\mathcal{S}_t$ based on the cosine similarity. These texts are concatenated into context C_k with $(1 - \theta) \cdot \lambda$ tokens. Finally, it returns $C_s \oplus C_k$ as the retrieved context for KET-RAG.

Notably, all chunks retrieved by KET-Retrieval, whether from entity or keyword channels, are fine-grained sub-chunks generated during the indexing stage through spitting and rewiring. This refinement reduces noise and preserves the context limit, allowing for the retrieval of more relevant knowledge during online queries.

5.3 Cost Analysis

We begin by analyzing the cost of KG-Index to construct $\mathcal{G} = (\mathcal{V}_e \cup \mathcal{V}_t, \mathcal{E})$. Let λ_e and λ_r denote the token counts of the prompt templates used to extract entities and relationships, respectively. Each text chunk node $v_i \in \mathcal{V}_t$ has a text of length ℓ. To extract entities, the LLM is prompted with $\ell + \lambda_e$ tokens for each v_i, resulting

Algorithm 4: KET-Retrieval $(\mathcal{G}, q, \lambda, \theta)$

Input: A TAG index $\mathcal{G} = \mathcal{G}_s \cup \mathcal{G}_k$, a query q, context length limit λ, retrieval ratio θ

Output: The context C

$C_s \leftarrow$ KG-Retrieval $(\mathcal{G}_s, \theta \cdot \lambda)$ in Algorithm 2;

$\mathcal{S}_k \leftarrow \arg\max_{v_i \in \mathcal{V}_k}$-k $\cos(v_i, q)$, s.t. $|\bigoplus(\mathcal{N}(\mathcal{S}_k))| = 2\cdot(1-\theta)\cdot\lambda$;

$\mathcal{S}_t \leftarrow \arg\max_{v_i \in \mathcal{N}(\mathcal{S}_k)}$-k $\cos(v_i, q)$, s.t. $|\bigoplus(\mathcal{S})| = (1-\theta)\cdot\lambda$;

$C_k \leftarrow \bigoplus(\mathcal{S})$;

return $C_s \oplus C_k$;

in $(\ell+\lambda_e)\cdot|\mathcal{V}_t|$ total input tokens. Similarly, extracting relationships requires $(\ell+\lambda_r)\cdot|\mathcal{V}_t|$ tokens. In addition, KG-Index must compute text embeddings for all nodes and edges, incurring another $\ell\cdot|\mathcal{V}_t| + \sum_{x\in\mathcal{V}_e\cup\mathcal{E}} \ell_x$ tokens, where ℓ_x is the description length of $x \in \mathcal{V}_e \cup \mathcal{E}$. Therefore, the total LLM Input Token Cost (ITC) for KG-Index is:

$$\mathrm{ITC}_{\text{KG-Index}} = \left(2 + \frac{(\lambda_e+\lambda_r)}{\ell}\right)\cdot\ell\cdot|\mathcal{V}_t|\cdot c_i + \left(\ell\cdot|\mathcal{V}_t| + \sum_{x\in\mathcal{V}_e\cup\mathcal{E}} \ell_x\right)\cdot c_e,$$

where c_i and c_e are the per-token costs for the LLM and embedding models, respectively. By contrast, KET-Index uses only a β-fraction of the KG-Index input token cost to construct $\mathcal{G}_s$. To build the text-keyword bipartite graph $\mathcal{G}_k$, it additionally consumes $3\ell\cdot|\mathcal{T}|$ tokens for multi-granular text embeddings (chunk, sub-chunk, and sentence levels). Thus,

$$\mathrm{ITC}_{\text{KET-Index}} = \beta\cdot\mathrm{ITC}_{\text{KG-Index}} + 3\cdot\ell\cdot|\mathcal{T}|\cdot c_e.$$

Regarding output token costs, KET-Index incurs only a β fraction of the output cost of KG-Index, as it generates only β of the entities and relations present in the full knowledge graph.

Regarding the retrieval and generation stages, all solutions, including KET-RAG, incur the same upper-bounded cost, as both stages are regulated by the maximum token parameter during LLM inference. Specifically, the input tokens for all solutions comprise a distinct prompt template and the retrieved content, which is constrained by the limit λ. The number of output tokens is then determined by subtracting the input token count from the maximum token limit, ensuring consistent computational costs across different approaches.

6 Experiments

In this section, we evaluate our proposed KET-RAG framework by addressing the following research questions:

- **RQ1**: How does KET-RAG enhance effectiveness while reducing costs compared to existing solutions?
- **RQ2**: What is the contribution of each core subroutine to KET-RAG's overall performance?
- **RQ3**: How does KET-RAG balance result quality, index construction cost, and the trade-off between its two retrieval channels?
- **RQ4**: How sensitive is KET-RAG's performance to its parameter settings?
- **RQ5**: How does KET-RAG 's retrieval latency compare to other baselines, and is there any compromise?

All experiments are conducted on a Linux machine with Intel Xeon(R) Gold 6240@2.60GHz CPU and 32GB RAM. We use the OpenAI API to access LLMs.

6.1 Experimental Settings

Datasets and metrics. We use two widely adopted benchmarking datasets for multi-hop QA tasks, MuSiQue [33] and HotpotQA [37], together with a benchmark RAG-QA Arena [15] for open-ended RAG evaluation. These datasets consist of QA pairs, each accompanied by multiple paragraphs as potential relevant context. Specifically, each QA pair is associated with 2 golden paragraphs out of 20 candidate paragraphs in MuSiQue, 2 golden paragraphs and 8 distracting paragraphs in HotpotQA, and one or more relevant paragraphs in RAG-QA Arena. Following prior work [14, 35], we sample 500 QA instances from each dataset. The corresponding paragraphs for all sampled instances are preprocessed into $\mathcal{T}$ and compiled as the external corpus for RAG [14]. The number of preprocessed paragraphs is 6,761 for MuSiQue (resp. 20,150 for HotpotQA and 7,010 for RAG-QA Arena), resulting in an overall token count of 751,784 (resp. 618,325 and 1,357,333). For MuSiQue and HotpotQA, we assess retrieval quality using *Coverage*, defined as the proportion of QA instances where the ground-truth answer is found within the retrieved context. Generation quality is evaluated by generating answers using a pre-defined LLM based *solely* on the retrieved context, and comparing these answers against ground truths using the following standard metrics [14, 21, 35, 38]: *Exact Match (EM)*, which measures the percentage of exact matches with the ground truth; *F1 score*, which captures partial correctness via word-level overlap; and *BERTScore*, which computes semantic similarity using BERT-based embeddings. For RAG-QA Arena, where answers are long-form, generation quality is evaluated using the Win Rate metric [15], defined as the proportion of cases in which the generated answer is preferred over a baseline answer (i.e., Text-RAG with $\ell = 1,200$) by an LLM. We use OpenAI's GPT-4o-mini as the judgment LLM.

Solutions and configurations. We evaluate the performance of 13 solutions categorized as follows: (i) *Existing competitors*: Text-RAG [19], KNNG-RAG [35], MS-Graph-RAG [8], Hybrid-RAG [32], HyDE [10], HippoRAG [14], LightRAG [13] including three retrieval frameworks LightRAG-Local, LightRAG-Global, and LightRAG-Hybrid. (ii) *Proposed baselines*: Keyword-RAG, which constructs only $\mathcal{G}_k$ as the index and retrieves from it; Skeleton-RAG, which retains only $\mathcal{G}_s$. (iii) *Final solutions*: KET-RAG-U and KET-RAG-P, where U and P represent selecting core chunks randomly and via PageRank, respectively. For a fair comparison, we implement most existing solutions within the KET-RAG framework, demonstrating that our approach serves as a unified and generalized RAG framework. Specifically, for Text-RAG and KNNG-RAG, we use the KNN graph constructed in Algorithm 3 as the index $\mathcal{G}$. KET-RAG reduces to Text-RAG when retrieving only the seed nodes in $\mathcal{G}$ and to KNNG-RAG when also including their neighbors. Furthermore, KET-RAG simplifies to MS-Graph-RAG by setting $\beta = 1$ and $\theta = 1$ and to Hybrid-RAG by combining both Text-RAG and MS-Graph-RAG. Notably, Hybrid-RAG fully constructs indices for both Text-RAG and MS-Graph-RAG, retrieving content equally from both. This setup effectively corresponds to $\beta = 0.5$ under a fixed context limit. For HyDE, we implement it by altering the generation

Table 2: Overall performance of RAG methods in low-cost versions. The best and second-best results in each column are highlighted in bold and underlined, respectively.

Dataset	MuSiQue					HotpotQA					RAG-QA Arena	
Metric	USD	Coverage	EM	F1	BERTScore	USD	Coverage	EM	F1	BERTScore	USD	Win Rate
Text-RAG	0.02	26.4	3.0	5.4	62.9	0.01	37.2	14.8	19.4	69.9	0.03	0.0
KNNG-RAG	0.02	21.2	2.8	4.6	62.5	0.01	28.6	13.4	16.9	68.8	0.03	17.8
MS-Graph-RAG	2.30	47.6	11.4	15.8	67.4	2.30	63.0	21.6	30.2	74.2	5.05	33.4
Hybrid-RAG	2.32	49.2	10.4	15.1	68.8	2.31	64.8	22.6	30.5	76.8	5.08	35.0
HyDE	0.03	33.0	4.2	6.7	63.5	0.02	40.8	16.2	21.3	70.9	0.03	33.2
HippoRAG	1.49	51.6	9.2	14.2	66.8	1.40	64.0	**29.2**	38.3	77.3	1.37	38.4
LightRAG-Local	1.80	39.0	9.4	12.9	66.6	1.77	57.6	22.4	28.1	73.1	4.00	29.0
LightRAG-Global	1.80	37.6	6.4	9.5	64.8	1.77	49.0	19.6	24.5	71.8	4.00	23.0
LightRAG-Hybrid	1.80	45.6	10.2	14.4	67.2	1.77	61.6	24.6	30.7	74.2	4.00	30.0
Keyword-RAG	0.03	50.8	7.0	11.8	68.1	0.03	60.2	24.4	33.5	**78.9**	0.07	37.0
Skeleton-RAG	1.86	43.4	11.0	14.1	66.8	1.84	57.8	20.0	26.7	73.2	4.04	28.6
KET-RAG-U	1.89	76.2	13.4	18.1	68.1	1.87	81.4	28.4	38.2	77.5	4.08	35.0
KET-RAG-P	1.89	**77.0**	**14.0**	**18.9**	**69.0**	1.87	**81.6**	28.6	**38.7**	77.5	4.08	**39.6**

prompt based on Text-RAG. Notably, for LightRAG and HippoRAG, we use their original code while aligning the same input parameters in subsequent experiments. Regarding the base models in each solution, we use OpenAI's `GPT-4o-mini` as the LLM for inference, OpenAI's `text-embedding-3-small` for text embedding generation, `cl100k_base` for word tokenization, and the `sent_tokenize` function from the `nltk` Python library for sentence tokenization. We follow the default settings in Edge et al. [8], setting the input chunk size ℓ to 1,200 and the output context limit λ to 12,000 across all solutions. Within KET-RAG, we use the default parameters for components related to KG-RAG and set $K = 2$, $\beta = 0.8$, and $\theta = 0.4$, unless specified otherwise. The implementations of all solutions are available at https://github.com/waetr/KET-RAG.

6.2 Performance Evaluation (RQ1)

In the first set of experiments, we evaluate the performance of KET-RAG against existing competitors under two configurations: a low-cost version with reduced accuracy and a high-accuracy version with increased cost. Following previous works [8], we achieve the low-cost setting by using an input chunk size of $\ell = 1,200$ and the high-accuracy setting by fixing $\ell = 150$ for all solutions. For Keyword-RAG and KET-RAG, we set τ to 3 and 0, respectively, ensuring the same text sub-chunk length in both configurations.

As reported in Tables 2-3, our proposed KET-RAG (-U/-P) achieves superior quality-cost trade-offs compared to existing methods across all datasets. In terms of retrieval quality, KET-RAG ranks as the best or second-best, achieving coverage scores of 77.0%/80.2% and 81.6%/82.6% on MuSiQue and HotpotQA, respectively. Compared to the competitor LightRAG (including LightRAG-Global, LightRAG-Local, and LightRAG-Hybrid), this corresponds to relative improvements of 68.8% and 32.5% on MuSiQue and HotpotQA, respectively. Most notably, we observe that KET-RAG in low-cost mode achieves comparable or even superior coverage to MS-Graph-RAG, HippoRAG, and LightRAG in high-accuracy mode while reducing indexing costs by over an order of magnitude. For example, on HotpotQA, the coverage scores of KET-RAG-P, MS-Graph-RAG, HippoRAG, and LightRAG are 81.6%, 74.6%, 79.6%, and 76.8%, respectively, yet KET-RAG-P incurs only 18.3% of their indexing cost. Akin to the retrieval quality, KET-RAG achieves competitive generation quality at lower costs. Take the low-cost mode as an example. It improves Hybrid-RAG by 37.2%/31.3%/0.3% (resp. 26.5%/26.9%/0.9%) in EM/F1 score/BERTScore on MuSiQue (resp. HotpotQA) while reducing indexing costs by 19%; on RAG-QA Arena, it improves the best competitor HippoRAG by 3.1%.

Regarding other competitors, we observe that Text-RAG exhibits a more pronounced accuracy improvement compared to MS-Graph-RAG, HippoRAG, and LightRAG when transitioning from low- to high-accuracy mode, which motivates the text splitting strategy in KET-RAG. Additionally, we find that, on HotpotQA, the generation quality of MS-Graph-RAG is slightly lower than that of Text-RAG in the high-accuracy setting. This is because HotpotQA is a weaker benchmark for multi-hop reasoning due to the presence of spurious signals [14, 33]. Despite this, KET-RAG consistently outperforms both competitors, demonstrating its robustness across different knowledge retrieval scenarios.

6.3 Ablation Study (RQ2)

In the second set of experiments, we evaluate the performance of each single building block proposed in KET-RAG, whose results are also included in Tables 2-3.

Knowledge graph skeleton. We evaluate the performance of Skeleton-RAG with its full version, MS-Graph-RAG. By default, Skeleton-RAG sets $\beta = 0.8$, resulting in a 20% reduction in indexing cost across all cases. Surprisingly, we find that Skeleton-RAG trades off only minor performance reductions, particularly in low-cost settings, while maintaining parity in high-accuracy configurations. For instance, in terms of EM score, Skeleton-RAG exhibits a relative decrease of 3.5% and 7.4% in the low-cost setting on MuSiQue and HotpotQA, respectively. However, in high-accuracy settings, there is no performance drop, and in the case of HotpotQA, even a

Table 3: Overall performance of RAG methods in high-performance versions. The best and second-best results in each column are highlighted in bold and underlined, respectively.

Dataset	MuSiQue					HotpotQA					RAG-QA Arena	
Metric	USD	Coverage	EM	F1	BERTScore	USD	Coverage	EM	F1	BERTScore	USD	Win Rate
Text-RAG	0.05	76.8	12.8	19.4	68.8	0.04	74.0	33.6	44.2	79.4	0.09	40.8
KNNG-RAG	0.05	66.0	11.2	17.5	68.7	0.04	63.2	29.8	39.6	78.1	0.09	35.6
MS-Graph-RAG	24.94	69.6	17.4	25.1	71.0	21.29	74.6	31.0	43.0	79.5	46.71	35.2
Hybrid-RAG	24.99	80.0	**19.4**	26.2	71.2	21.33	80.2	34.0	46.1	80.2	46.80	39.4
HyDE	0.05	76.8	15.4	22.5	70.3	0.05	78.8	**35.2**	45.5	80.3	0.13	43.0
HippoRAG	8.70	**81.4**	14.8	19.8	69.0	10.19	79.6	31.8	43.8	79.7	12.21	39.6
LightRAG-Local	*	*	*	*	*	11.06	76.0	31.0	43.6	79.7	29.24	38.4
LightRAG-Global	*	*	*	*	*	11.06	63.0	20.8	30.0	74.6	29.24	38.2
LightRAG-Hybrid	*	*	*	*	*	11.06	76.8	29.8	40.9	78.8	29.24	39.8
Keyword-RAG	0.09	78.2	14.6	20.6	69.3	0.07	82.2	33.8	46.4	80.8	0.15	41.6
Skeleton-RAG	19.95	69.6	17.4	24.6	70.7	17.03	74.6	31.2	42.8	79.3	37.39	34.4
KET-RAG-U	20.04	80.2	18.4	25.9	71.0	17.10	**82.6**	34.8	47.2	81.4	37.53	38.0
KET-RAG-P	20.04	79.6	19.2	**26.6**	**71.3**	17.10	**82.6**	**35.2**	**47.7**	**81.5**	37.53	**43.2**

* We exclude the results of LightRAG on MuSiQue due to unexpected behavior observed when running its original implementation.

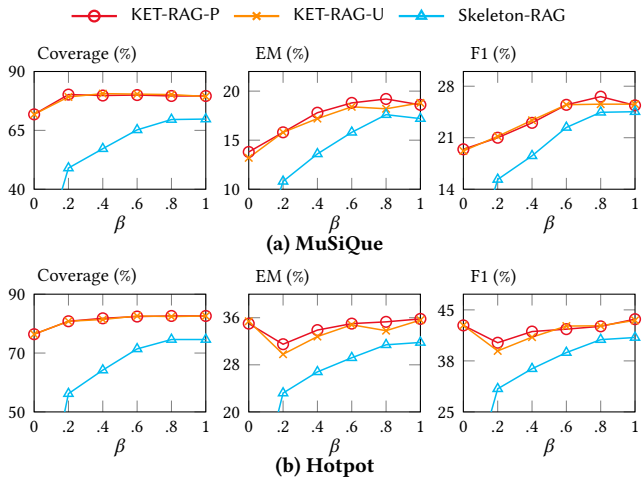


Figure 3: Answer quality by varying β.

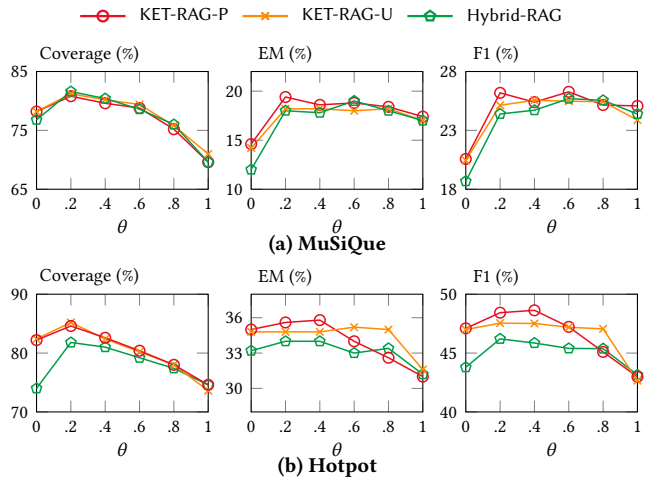


Figure 4: Answer quality by varying θ.

slight improvement is observed. These results suggest that Skeleton-RAG effectively balances efficiency and quality, making it a viable alternative to full-scale knowledge graph indexing.

Text-keyword bipartite graph. We compare Keyword-RAG with the conventional Text-RAG. To ensure a fair comparison, we set $\tau = 0$ for Keyword-RAG, allowing both Keyword-RAG and Text-RAG to retrieve text chunks of the same size. As shown in Tables 2-3, Keyword-RAG consistently outperforms Text-RAG in retrieval and generation quality. Notably, in the low-cost setting, Keyword-RAG achieves 92.4%/133.3%/118.5% and 61.8%/52.5%/58.8% relative improvement in Coverage/EM/F1 on MuSiQue and HotpotQA, respectively, with more significant gains on MuSiQue. It demonstrates the effectiveness of retrieving context from neighboring text chunks of keyword seeds, particularly in complex multi-hop reasoning.

Core text chunk identification. Based on the results in Tables 2-3, we observe that KET-RAG-P shows better quality than KET-RAG-U in both the low-cost and high-cost modes. For instance, KET-RAG-P outperforms KET-RAG-U by up to 1.0%/4.5%/4.4% in Coverage/EM/F1 scores on MuSiQue. This confirms the effectiveness of the core chunk identification technique, as motivated in Section 4.2. Additionally, we observe that the superiority of KET-RAG-P remains consistent across different settings of β and θ, as further illustrated in Figures 3–4.

6.4 Trade-off Analysis (RQ3)

In the third set of experiments, we analyze the trade-off between accuracy and cost by varying the budget β, as well as the balance between the two retrieval channels by adjusting θ. We set $\ell = 150$ and $\tau = 0$, and follow the default parameter settings in Section 6.1.

Table 4: Answer quality by varying ℓ and τ on MuSiQue.

Param	ℓ				τ			
Value	**150**	**300**	**600**	**1200**	**3**	**2**	**1**	**0**
Coverage	79.6	79.6	77.8	77.0	77.0	70.4	61.0	56.8
EM	19.2	18.8	15.4	14.0	14.0	13.8	11.8	12.8
F1	22.3	18.8	17.7	17.2	18.9	18.3	16.3	17.2

Table 5: Answer quality by varying K on MuSiQue.

K	**2**	**4**	**10**
Coverage/EM/F1	79.6/19.2/26.1	80.0/17.8/25.2	80.4/19.4/26.0

Table 6: Retrieval time (low cost/high perf.) in seconds.

Method	MuSiQue	HotpotQA	RAG-QA Arena
Text-RAG	0.03/0.03	0.03/0.02	0.03/0.02
KNNG-RAG	0.03/0.23	0.03/0.18	0.04/0.38
MS-Graph-RAG	0.43/1.99	0.39/1.75	0.46/2.14
Hybrid-RAG	0.41/1.07	0.40/0.93	0.43/1.26
HyDE	0.04/0.03	0.03/0.03	0.04/0.02
Keyword-RAG	0.03/0.04	0.04/0.04	0.04/0.05
Skeleton-RAG	0.31/1.64	0.28/1.51	0.32/1.68
KET-RAG-U	0.28/0.79	0.29/0.63	0.28/0.87
KET-RAG-P	0.29/0.78	0.40/0.87	0.28/0.88

Accuracy-cost tradeoff. Figure 3 presents the performance of KET-RAG-U, KET-RAG-P, and Skeleton-RAG on MuSiQue and HotpotQA by varying β. In particular, KET-RAG-P and KET-RAG-U consistently outperform Skeleton-RAG, which serves as a lower bound, across different budget β values. Between the two variants, KET-RAG-P achieves better performance than KET-RAG-U particularly when $\beta \in [0.6, 0.8]$ in MuSiQue and $\beta \in [0.2, 0.4]$ in MuSiQue, demonstrating the effectiveness of identifying core text chunks using PageRank centralities. Furthermore, the performance of KET-RAG is less sensitive to variations in β on HotPotQA. For instance, the coverage at $\beta = 0.2$ decreases by 2% compared to $\beta = 1$. On MuSiQue, although KET-RAG exhibits greater sensitivity in generation quality, its performance quickly catches up once β reaches 0.6. These findings demonstrate KET-RAG's effectiveness for further reducing indexing costs.

Retrieval channel. Figure 4 reports the performance of KET-RAG-U, KET-RAG-P, and Hybrid-RAG by varying θ. As illustrated, KET-RAG consistently outperforms Hybrid-RAG across different θ settings, demonstrating the effectiveness of its two key components, Keyword-RAG and Skeleton-RAG. Additionally, we observe that the performance of Keyword-RAG improves significantly when incorporating a small fraction (e.g., 0.2) of context from MS-Graph-RAG. This finding further motivates the reduction of costs associated with building a full knowledge graph. Regarding the two variants, KET-RAG-U and KET-RAG-P, we find that KET-RAG-P achieves superior EM and F1 scores when $\theta \leq 0.4$, which aligns with the trend observed in Figure 3.

6.5 Parameter Sensitivity (RQ4)

In the fourth set of experiments, we take the Musique dataset as an example and analyze the sensitivity of KET-RAG-P w.r.t. the input text chunk size ℓ, the number of splits τ, and the integer K used for KNN graph construction. For ℓ, we vary $\ell = 150, 300, 600, 1200$ and set the corresponding $\tau = 0, 1, 2, 3$ to maintain a consistent sub-chunk length. For τ, we fix $\ell = 1200$ and vary $\tau = 0, 1, 2, 3$. Additionally, we set $K = 2, 4, 10$ to explore different KNN graph densities. As shown in Table 4, KET-RAG-P achieves better retrieval and generation quality as the size of input chunks or split sub-chunks decreases. This trend is consistent with the performance of Keyword-RAG and Graph-RAG in both low- and high-cost settings. These findings align with previous observations [8] that smaller chunk sizes improve result quality, further validating the text chunk splitting design in KET-RAG. Additionally, as shown in Table 5, varying the integer K from 2 to 10 results in only minor changes in coverage, EM, and F1 scores, e.g., 79.6/19.2/26.1 for $K = 2$ vs. 80.4/19.4/26.0 for $K = 10$, indicating that KET-RAG is not significantly affected by the density of the KNN graph. This stability can be explained by Figure 2, which shows that KNN graphs with different K exhibit similar degree distribution shapes, despite variations in average degree.

6.6 Retrieval Latency (RQ5)

In the fifth set of experiments, we report the average retrieval latency per query. To reduce retrieval latency, our framework employs concurrent retrieval using multithreading, with the maximum thread count set to 30. We report the retrieval latency of KET-RAG and all other methods implemented within the same unified framework in Table 6. Specifically, KET-RAG achieves average retrieval times of 0.39×/0.73× those of MS-Graph-RAG and Hybrid-RAG on MuSiQue, 0.43×/0.80× on HotpotQA, and 0.41×/0.70× on RAG-QA Arena. Although Text-RAG is the fastest due to its simplicity, KET-RAG's retrieval latency remains under one second. In the low-cost setting, all graph-based RAG variants complete retrieval within 0.5 seconds. LLM generation time is consistent across methods, averaging 0.42 seconds per query. Overall, there is *no significant compromise* in latency when using KET-RAG, especially given its improved retrieval quality.

7 Conclusions

In this work, we propose KET-RAG, a cost-efficient multi-granular indexing framework for Graph-RAG systems. By integrating a knowledge graph skeleton with a text-keyword bipartite graph, KET-RAG improves retrieval and generation quality while significantly reducing indexing costs. Our approach matches or surpasses Microsoft's Graph-RAG in retrieval quality while reducing indexing costs by over an order of magnitude, and improves generation quality by up to 32.4% with 20% lower indexing costs. For future work, we plan to extend KET-RAG to global search scenarios and explore adaptive paradigms for real-world scalable deployment.

Acknowledgments

This research is supported by the Ministry of Education, Singapore, under its MOE AcRF TIER 3 Grant (MOE-MOET32022-0001).

References

[1] Mohannad Alhanahnah, Yazan Boshmaf, and Benoit Baudry. 2024. DepesRAG: Towards Managing Software Dependencies using Large Language Models. *arXiv preprint arXiv:2405.20455* (2024).

[2] Shawn Arnold and Clayton Romero. 2022. The Vital Role of Managing e-Discovery. https://legal-tech.blog/the-vital-role-of-managing-e-discovery

[3] Lele Cao, Vilhelm von Ehrenheim, Mark Granroth-Wilding, Richard Anselmo Stahl, Andrew McCornack, Armin Catovic, and Dhiana Deva Cavalcanti Rocha. 2024. CompanyKG: A Large-Scale Heterogeneous Graph for Company Similarity Quantification. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4816–4827.

[4] Xuanzhong Chen, Xiaohao Mao, Qihan Guo, Lun Wang, Shuyang Zhang, and Ting Chen. 2024. RareBench: Can LLMs Serve as Rare Diseases Specialists?. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4850–4861.

[5] Andrea Colombo. 2024. Leveraging Knowledge Graphs and LLMs to Support and Monitor Legislative Systems. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 5443–5446.

[6] Mohammad Dehghan, Mohammad Alomrani, Sunyam Bagga, David Alfonso-Hermelo, Khalil Bibi, Abbas Ghaddar, Yingxue Zhang, Xiaoguang Li, Jianye Hao, Qun Liu, Jimmy Lin, Boxing Chen, Prasanna Parthasarathi, Mahdi Biparva, and Mehdi Rezagholizadeh. 2024. EWEK-QA : Enhanced Web and Efficient Knowledge Graph Retrieval for Citation-based Question Answering Systems. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*.

[7] Julien Delile, Srayanta Mukherjee, Anton Van Pamel, and Leonid Zhukov. 2024. Graph-Based Retriever Captures the Long Tail of Biomedical Knowledge. *arXiv preprint arXiv:2402.12352* (2024).

[8] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. 2024. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130* (2024).

[9] Wenqi Fan, Yujuan Ding, Liangbo Ning, Shijie Wang, Hengyun Li, Dawei Yin, Tat-Seng Chua, and Qing Li. 2024. A Survey on RAG Meeting LLMs: Towards Retrieval-Augmented Large Language Models. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 6491–6501.

[10] Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. 2023. Precise zero-shot dense retrieval without relevance labels. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 1762–1777.

[11] Ant Group and OpenKG. 2023. Semantic-enhanced Programmable Knowledge Graph (SPG) White paper (v1.0). https://spg.openkg.cn/en-US.

[12] Yu Gu, Robert Tinn, Hao Cheng, Michael Lucas, Naoto Usuyama, Xiaodong Liu, Tristan Naumann, Jianfeng Gao, and Hoifung Poon. 2021. Domain-specific language model pretraining for biomedical natural language processing. *ACM Transactions on Computing for Healthcare (HEALTH)* 3, 1 (2021), 1–23.

[13] Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. 2025. LightRAG: Simple and Fast Retrieval-Augmented Generation. arXiv:2410.05779 [cs.IR] https://arxiv.org/abs/2410.05779

[14] Bernal Jimenez Gutierrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*. https://openreview.net/forum?id=hkujvAPVsg

[15] Rujun Han, Yuhao Zhang, Peng Qi, Yumo Xu, Jenyuan Wang, Lan Liu, William Yang Wang, Bonan Min, and Vittorio Castelli. 2024. RAG-QA Arena: Evaluating Domain Robustness for Long-form Retrieval Augmented Question Answering. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 4354–4374. doi:10.18653/v1/2024.emnlp-main.249

[16] Xiaoxin He, Yijun Tian, Yifei Sun, Nitesh V Chawla, Thomas Laurent, Yann LeCun, Xavier Bresson, and Bryan Hooi. 2024. G-retriever: Retrieval-augmented generation for textual graph understanding and question answering. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.

[17] Bowen Jin, Chulin Xie, Jiawei Zhang, Kashob Kumar Roy, Yu Zhang, Zheng Li, Ruirui Li, Xianfeng Tang, Suhang Wang, Yu Meng, et al. 2024. Graph chain-of-thought: Augmenting large language models by reasoning on graphs. *arXiv preprint arXiv:2404.07103* (2024).

[18] Rishi Kalra, Zekun Wu, Ayesha Gulley, Airlie Hilliard, Xin Guan, Adriano Koshiyama, and Philip Treleaven. 2024. HyPA-RAG: A Hybrid Parameter Adaptive Retrieval-Augmented Generation System for AI Legal and Policy Applications. In *Proceedings of the 1st Workshop on Customizable NLP: Progress and Challenges in Customizing NLP for a Domain, Application, Group, or Individual (CustomNLP4U)*.

[19] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems* 33 (2020), 9459–9474.

[20] Dawei Li, Shu Yang, Zhen Tan, Jae Young Baik, Sukwon Yun, Joseph Lee, Aaron Chacko, Bojian Hou, Duy Duong-Tran, Ying Ding, Huan Liu, Li Shen, and Tianlong Chen. 2024. DALK: Dynamic Co-Augmentation of LLMs and KG to answer Alzheimer's Disease Questions with Scientific Literature. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 2187–2205.

[21] Zijian Li, Qingyan Guo, Jiawei Shao, Lei Song, Jiang Bian, Jun Zhang, and Rui Wang. 2024. Graph Neural Network Enhanced Retrieval for Question Answering of LLMs. *arXiv preprint arXiv:2406.06572* (2024).

[22] Costas Mavromatis and George Karypis. 2024. GNN-RAG: Graph Neural Retrieval for Large Language Model Reasoning. *arXiv preprint arXiv:2405.20139* (2024).

[23] Sewon Min, Danqi Chen, Luke Zettlemoyer, and Hannaneh Hajishirzi. 2019. Knowledge guided text retrieval and reading for open domain question answering. *arXiv preprint arXiv:1911.03868* (2019).

[24] Xinyi Mou, Zejun Li, Hanjia Lyu, Jiebo Luo, and Zhongyu Wei. 2024. Unifying Local and Global Knowledge: Empowering Large Language Models as Political Experts with Knowledge Graphs. In *Proceedings of the ACM on Web Conference 2024*. 2603–2614.

[25] Sai Munikoti, Anurag Acharya, Sridevi Wagle, and Sameera Horawalavithana. 2024. ATLANTIC: Structure-Aware Retrieval-Augmented Language Model for Interdisciplinary Science. *Proceedings of the Workshop on AI to Accelerate Science and Engineering (AI2ASE). Held in conjunction with the 38th AAAI Conference on Artificial Intelligence*. (2024).

[26] NebulaGraph. 2023. NebulaGraph Launches Industry-First Graph RAG: Retrieval-Augmented Generation with LLM Based on Knowledge Graphs. https://www.nebula-graph.io/posts/graph-RAG.

[27] Neo4j. 2023. NaLLM. https://github.com/neo4j/NaLLM.

[28] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. 1999. *The PageRank citation ranking: Bringing order to the web.* Technical Report. Stanford InfoLab.

[29] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. 2024. Graph Retrieval-Augmented Generation: A Survey. *arXiv preprint arXiv:2408.08921* (2024).

[30] Zhuoyi Peng and Yi Yang. 2024. Connecting the Dots: Inferring Patent Phrase Similarity with Retrieved Phrase Graphs. In *Findings of the Association for Computational Linguistics: NAACL 2024*.

[31] Juan Ramos et al. 2003. Using tf-idf to determine word relevance in document queries. In *Proceedings of the first instructional conference on machine learning*, Vol. 242. Citeseer, 29–48.

[32] Bhaskarjit Sarmah, Benika Hall, Rohan Rao, Sunil Patel, Stefano Pasquali, and Dhagash Mehta. 2024. HybridRAG: Integrating Knowledge Graphs and Vector Retrieval Augmented Generation for Efficient Information Extraction. *arXiv preprint arXiv:2408.04948* (2024).

[33] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. MuSiQue: Multihop Questions via Single-hop Question Composition. *Transactions of the Association for Computational Linguistics* 10 (2022), 539–554.

[34] Ruijie Wang, Zheng Li, Danqing Zhang, Qingyu Yin, Tong Zhao, Bing Yin, and Tarek Abdelzaher. 2022. RETE: retrieval-enhanced temporal event forecasting on unified query product evolutionary graph. In *Proceedings of the ACM Web Conference 2022*. 462–472.

[35] Yu Wang, Nedim Lipka, Ryan A Rossi, Alexa Siu, Ruiyi Zhang, and Tyler Derr. 2024. Knowledge graph prompting for multi-document question answering. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 19206–19214.

[36] Zhentao Xu, Mark Jerome Cruz, Matthew Guevara, Tie Wang, Manasi Deshpande, Xiaofeng Wang, and Zheng Li. 2024. Retrieval-augmented generation with knowledge graphs for customer service question answering. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2905–2909.

[37] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. 2369–2380.

[38] Hao Yu, Aoran Gan, Kai Zhang, Shiwei Tong, Qi Liu, and Zhaofeng Liu. 2024. Evaluation of retrieval-augmented generation: A survey. In *CCF Conference on Big Data*. Springer, 102–120.

[39] Yingli Zhou, Yaodong Su, Youran Sun, Shu Wang, Taotao Wang, Runyuan He, Yongwei Zhang, Sicong Liang, Xilin Liu, Yuchi Ma, et al. 2025. In-depth Analysis of Graph-based RAG in a Unified Framework. *arXiv preprint arXiv:2503.04338* (2025).